

编译设计文档汇总

本次作业当中能采用的是 C++和 Java。其中我采用的是C++。

1. 词法分析设计

本次作业的主要目的是设计并实现一个能够进行词法分析的程序。该程序能够识别源代码中的单词，记录其类别和具体值，并将结果输出到指定文件中。此外，程序还需能够检测源代码中的词法错误，并准确报告错误的行号和类别码，以支持后续的语法和语义分析。

这问题要求设计并实现一个编译器的词法分析程序。该词法分析程序遵循一下规则：

单词名称	类别码	单词名称	类别码	单词名称	类别码	单词名称	类别码
Ident	IDENFR	else	ELSETK	void	VOIDTK	；	SEMICN
IntConst	INTCON	！	NOT	*	MULT	，	COMMA
StringConst	STRCON	&&	AND	/	DIV	(	LPARENT
CharConst	CHRCON		OR	%	MOD	)	RPARENT
main	MAINTK	for	FORTK	<	LSS	[	LBRACK
const	CONSTTK	getint	GETINTTK	<=	LEQ	]	RBRACK
int	INTTK	getchar	GETCHARTK	>	GRE	{	LBRACE
char	CHARTK	printf	PRINTFTK	>=	GEQ	}	RBRACE
break	BREAKTK	return	RETURNTK	==	EQL		
continue	CONTINUETK	+	PLUS	!=	NEQ		
if	IFTK	-	MINU	=	ASSIGN		

程序需要从

`testfile.txt` 中读取源代码，识别并输出词法单元（包括类别码和字符形式）到 `lexer.txt`，或者在有错误时输出错误行号和错误类别码到 `error.txt`。

定义枚举类型 `TokenType`

首先，定义一个名为 `TokenType` 的枚举类型，用于标识每个词法单元的类型。

```
enum TokenType {
    IDENFR, INTCON, STRCON, CHRCON, MAINTK, CONSTTK, INTTK, CHARTK, BREAKTK,
    CONTINUETK, IFTK, ELSETK, FORTK, GETINTTK, GETCHARTK, PRINTFTK, RETURNTK,
    VOIDTK, NOT, AND, OR, MULT, DIV, MOD, LSS, LEQ, GRE, GEQ, EQL, NEQ, ASSIGN,
    PLUS, MINU, SEMICN, COMMA, LPARENT, RPARENT, LBRACK, RBRACK, LBRACE, RBRACE
};
```

使用

`unordered_map` 定义关键字和操作符

为了提高查找效率，使用 `unordered_map` 来存储关键字和操作符。

```
unordered_map<string, TokenType> keywords = {
    {"const", CONSTTK}, {"int", INTTK}, {"char", CHARTK}, {"break", BREAKTK},
    {"continue", CONTINUETK}, {"if", IFTK}, {"else", ELSETK}, {"for", FORTK},
    {"getint", GETINTTK}, {"getchar", GETCHARTK}, {"printf", PRINTFTK},
    {"return", RETURNTK}, {"void", VOIDTK}, {"main", MAINTK}
};

unordered_map<string, TokenType> operators = {
    {"+", PLUS}, {"-", MINU}, {"*", MULT}, {"/", DIV}, {"%", MOD},
    {"<", LSS}, {"<=", LEQ}, {">", GRE}, {">=", GEQ}, {"==", EQL},
    {"!=", NEQ}, {"=", ASSIGN}, {"&&", AND}, {"||", OR}, {"!", NOT},
    {";", SEMICN}, {"", COMMA}, {"(", LPARENT}, {")", RPARENT},
    {"[", LBRACK}, {"]", RBRACK}, {"{", LBRACE}, {"}", RBRACE}
};
```

→ `unordered_map` 的设计主要是为了通过哈希表实现快速查找关键字和操作符，简化代码结构，提升查找效率和代码的可维护性。

实现 `tokenTypeToString` 函数

```
string tokenTypeToString(TokenType type) {
    switch (type) {
        case IDENFR: return "IDENFR";
        case INTCON: return "INTCON";
        case STRCON: return "STRCON";
        // ... (将所有 TokenType 转换为对应的字符串)
        default: return "UNKNOWN";
    }
}
```

主程序逻辑

初始化

```
ifstream inputFile("testfile.txt");
char currentChar;
int lineNumber = 1;
```

初始化之后，我最先考虑的是处理空白字符和换行。

```
if (isspace(currentChar))
{
    if (currentChar == '\n')
    {
        lineNumber++;
    }
    continue;
}
```

为了方便处理error，出现错误时执行 `lineNumber++`。且遇到换行符号时，增加行号计数。

然后，考虑的时，处理注释部分。

```
else if(currentChar == '/')
{
    char nextChar = inputFile.peek();

    // 处理多行注释
    if(nextChar == '*')
    {
        // 代码略
        continue;
    }

    // 处理单行注释
    else if(nextChar == '/')
    {
        // 代码略
        continue;
    }

    // 处理除号
    else
    {
        tokens.push_back({DIV, "/", lineNumber});
    }
}
```

注释部分会分成3个部分。一是处理多行注释，二是单行注释，还有可能是除号。所以分为3个部分了。其中，`continue` 的意思是跳过当前循环的剩余部分，直接开始下一次循环，也就是说，当 `continue` 执行后，下面的代码将不再执行，程序会进入下一次循环。

以上两个部分是算一种特殊情况。处理完这种特殊情况之后，得要考虑如何处理词法分析程。

我是以 `is` 开头的函数 来判断，来检查当前字符是否为字母、数字或其他特定类型。

- `isalpha(char)`：判断字符是否为字母，如果是字母返回 `true`，否则返回 `false`。
- `isalnum(char)`：判断字符是否为字母或数字，如果是字母或数字返回 `true`，否则返回 `false`。
- `isdigit(char)`：判断字符是否为数字，如果是数字返回 `true`，否则返回 `false`。
- `isspace(char)`：判断字符是否为空白字符（如空格、制表符、换行符等），如果是返回 `true`，否则返回 `false`。

其中，重要看的部分是以下部分：

```
else if (currentChar == '\\')
{
    string charConst;
    charConst += currentChar;
    char c = inputFile.get();
    charConst += c;

    if (c == '\\')
    {
        char nextChar = inputFile.get();
        charConst += nextChar;
        c = nextChar;
    }

    if (inputFile.peek() == '\\')
    {
        charConst += inputFile.get();
        tokens.push_back({CHRCON, charConst, lineNumber});
    }

    else
    {
        errors.push_back({lineNumber, 'a'});
    }
}
```

这个代码段用于处理字符常量。当检测到当前字符是单引号 `'` 时，程序开始构建字符常量字符串，首先读取下一个字符 `c`。如果 `c` 是反斜杠 `\\`，则表示这是一个转义字符，需要再读取一个字符并追加到字符常量中。随后，检查下一个字符是否是关闭的单引号 `'`，如果是，则将完整的字符常量作为 `CHRCON` 类型的词法单元加入到 `tokens` 列表中；如果不是，则记录错误信息，表示字符常量未正确闭合。

2. 语法分析

本次作业的主要目的是设计并实现一个能够进行语法分析的程序。该程序使用递归下降方法，能够识别源代码中的语法成分，记录并输出其结构信息。此外，程序还需检测源代码中的语法错误，并准确报告错误的行号和类别码，以支持后续的语义分析。

由于这次作业不必要考虑语义，直接翻译既可。

根据递归向下分析法，我设计并实现了每个函数：

```
void lexicalAnalysis(const string& filename);          // 语义部分
void outputToken(const Token& token);
void advanceToken();
void parseExp();
void parseNumber();
void parseCharacter();
```

```

void parseLVal();
void parsePrimaryExp();
void parseFuncRParams();
void parseCond();
void parseUnaryOp();
void parseUnaryExp();
void parseConstExp();
void parseMulExp();
void parseAddExp();
void parseRelExp();
void parseEqExp();
void parseLAndExp();
void parseLOrExp();
void parseForStmt();
void parseStmt();
void parseFuncType();
void parseBType();
void parseConstDecl();
void parseConstDef();
void parseFuncFParam();
void parseFuncDef();
void parseFuncFParams();
void parseMainFuncDef();
void parseConstInitVal();
void parseVarDecl();
void parseVarDef();
void parseInitVal();
void parseBlockItem();
void parseBlock();
void parseDecl();
void parseCompUnit();

```

```

// ;
void parseErrorPrintfI() {
    Token currentToken = getCurrentToken();
    Token previousToken = getPreviousToken();

    if (currentToken.type == SEMICN) {
        outputToken(currentToken);
        advanceToken();
    }
    else {
        errors.push_back({ previousToken.lineNumber, 'i' });
    }
}

```

其中，我做了额外的输出错误部分和完善了错误检测机制。在语法分析过程中，当检测到缺失分号(;)、右括号(>)或右中括号(])等语法错误时，程序会调用相应的错误处理函数（如 `parseErrorPrintfI`、`parseErrorPrintfJ` 和 `parseErrorPrintfK`）来记录错误信息。这些错误信息包括错误所在的行号和错误类别码，并将其存储在一个错误列表中。

```

Token getPreviousToken()
{
    if (tokenIndex > 0 && tokenIndex - 1 < tokens.size())
        return tokens[tokenIndex - 1];
    else
        return Token{ IDENFR, "", 0 };
}

```

在 `else` 处理过程中，如果执行 `advanceToken()`，会导致标记（token）移动两次，从而多次输出重复的错误信息。因此，不调用 `advanceToken()` 解决了标记移动两次的问题。还有，为了处理前一次词法分析中缺失分号导致解析器自动跳到下一行的问题，我设计了 `getPreviousToken()` 函数。由于在

缺少分号的情况下，词法分析器会跳过当前行并继续解析下一行，这可能会导致语法分析器无法准确定位错误位置。

我觉得本次作业的最难点在于处理 `parseStmt()` 函数的处理。

```
语句 Stmt → LVal '=' Exp ';' // 每种类型的语句都要覆盖
| [Exp] ';' //有无Exp两种情况
| Block
| 'if' '(' Cond ')' Stmt [ 'else' Stmt ] // 1.有else 2.无else
| 'for' '(' [ForStmt] ';' [Cond] ';' [ForStmt] ')' Stmt // 1. 无缺省, 1种情况 2.
ForStmt与Cond中缺省一个, 3种情况 3. ForStmt与Cond中缺省两个, 3种情况 4. ForStmt与Cond全部
缺省, 1种情况
| 'break' ';' | 'continue' ';'
| 'return' [Exp] ';' // 1.有Exp 2.无Exp
| LVal '=' 'getint' '(' ')' ';'
| LVal '=' 'getchar' '(' ')' ';'
| 'printf' '(' StringConst {'', 'Exp'} ')' ';' // 1.有Exp 2.无Exp
```

以下是

`parseStmt()` 代码中 `else` 块部分的详细解释：

```
// LVal '=' Exp ';' 或者 LVal '=' 'getint' '(' ')' ';' 或者 'getchar' '(' ')' ';'
else {
    int index = 0;
    bool isLVal = false;

    // 判断是否为 LVal (通过检查下一个 token 是否为 '=')
    if (tokenIndex + 1 < tokens.size()) {
        Token nextToken = tokens[tokenIndex + 1];

        while (true) {
            if (nextToken.type == ASSIGN) {
                isLVal = true;
                break;
            }

            if (nextToken.type == SEMICN || tokenIndex + index >= tokens.size()) {
                break;
            }
            index++;
            nextToken = tokens[tokenIndex + index];
        }
    }

    // 如果是 LVal, 则进行相应处理
    if (isLVal) {
        parseLVal(); // 处理 LVal

        // 输出 '=' 并移动到下一个 token
        outputToken(getCurrentToken());
        advanceToken();
        Token token = getCurrentToken();

        // 处理 'getint' 或 'getchar'
        if (token.type == GETINTTK || token.type == GETCHARTK) {
            outputToken(token);
            advanceToken();

            // 处理 '('
            if (getCurrentToken().type == LPARENT) {
                outputToken(getCurrentToken());
                advanceToken();
            }
        }
    }
}
```

```

        // 处理 ')'
        if (getCurrentToken().type == RPARENT) {
            outputToken(getCurrentToken());
            advanceToken();
        }
        else {
            parseErrorPrintfJ();
        }

        // 处理 ';'
        if (getCurrentToken().type == SEMICN) {
            outputToken(getCurrentToken());
            advanceToken();
        }
        else {
            parseErrorPrintfI();
        }
    }

    // 处理 LVal '=' Exp ';'
    else {
        parseExp();

        // 处理 ';'
        if (getCurrentToken().type == SEMICN) {
            outputToken(getCurrentToken());
            advanceToken();
        }
        else {
            parseErrorPrintfI();
        }
    }
}

// 处理 [Exp] ';'
else {
    if (getCurrentToken().type != SEMICN) {
        parseExp();
    }

    // 处理 ';'
    if (getCurrentToken().type == SEMICN) {
        outputToken(getCurrentToken());
        advanceToken();
    }
    else {
        parseErrorPrintfI();
    }
}
}
}

```

在 `parseStmt()` 函数的 `else` 分支中必须单独处理 `LVal`，这是因为 `LVal` 可以出现在多种不同的语句中，如赋值语句和函数调用赋值，每种形式需要不同的解析逻辑。解析 `LVal` 时需要进行前瞻分析，以确定下一个 `token` 是否为赋值运算符，这增加了代码的复杂性。赋值语句不仅包括简单的 `LVal = Exp;`，还涉及 `LVal = getint();` 和 `LVal = getchar();` 等多种情况，每种情况都需要独立处理。`LVal` 的多样使用场景导致 `else` 分支中需要处理大量可能性，使代码难以维护。此外，解析 `LVal` 时可能遇到多种语法错误，每种错误情况都需要单独的处理逻辑，进一步增加了实现的复杂性。

并且在我的代码中，`advanceToken` 函数只是简单地增加了标记（`token`）的++，并且存在无法正确读取下一个标记的问题。因此，在大多数情况下，我使用 `getCurrentToken()` 来直接读取下一个标记。

错误的处理过程当中，面临了以下的问题：

```

int main(){
    char c;

```

```

    c = getchar()
    if(1 & 2){
        printf("output is %c\n", c);
    }
    return 0;
}
```

```

3 i
4 a
```

误类型	错误类别码	解释	对应文法及出错符号
非法符号	a	出现了 '&' 和 ' ' 这两个符号，应该将其当做 '&&' 与 ' ' 进行处理，但是在记录单词名称的时候仍记录 '&' 和 ' '，报错行号为 '&' 或 ' ' 所在的行号	LAndExp $\rightarrow$ LAndExp '&&' EqExp LOrExp $\rightarrow$ LOrExp ' ' LAndExp
名字重定义	b	函数名或者变量名在当前作用域下重复定义。注意，变量一定是同一级作用域下才会判定出错，不同级作用域下，内层会覆盖外层定义。报错行号 Ident 所在行数。	ConstDef $\rightarrow$ Ident, VarDef $\rightarrow$ Ident ... Ident, FuncDef $\rightarrow$ FuncType Ident ...
未定义的名字	c	使用了未定义的标识符报错行号为 Ident 所在行数。	LVal $\rightarrow$ Ident ..., UnaryExp $\rightarrow$ Ident ...
函数参数个数不匹配	d	函数调用语句中，参数个数与函数定义中的参数个数不匹配。报错行号为函数调用语句的函数名所在行数。	UnaryExp $\rightarrow$ Ident '(', [FuncRParams] ')'
函数参数类型不匹配	e	函数调用语句中，参数类型与函数定义中对对应位置的参数类型不匹配。报错行号为函数调用语句的函数名所在行数。关于参数类型不匹配会在下方特别说明进行详细说明。	UnaryExp $\rightarrow$ Ident '(', [FuncRParams] ')'
无返回值的函数存在不匹配的 return 语句	f	报错行号为‘return’所在行号。	Stmt $\rightarrow$ 'return' '{' '[Exp]' '}' ';'
有返回值的函数缺少 return 语句	g	只需要考虑函数末尾是否存在 return 语句，无需考虑数据流。报错行号为函数结尾的‘}’所在行号。	FuncDef $\rightarrow$ FuncType Ident '(', [FuncFParams] ')' Block
不能改变常量的值	h	LVal 为常量时，不能对其修改。报错行号为 LVal 所在行号。	Stmt $\rightarrow$ LVal '=' Exp ';', Stmt $\rightarrow$ LVal '=' 'getint' '(' ')' ';'
缺少分号	i	报错行号为分号前一个非终结符所在行号。	Stmt, ConstDecl 和 VarDecl 中的 ';'
缺少右小括号 ')'	j	报错行号为右小括号前一个非终结符所在行号。	函数调用、函数定义、Stmt、PrimaryExp 中的 ')'
缺少右中括号 ']'	k	报错行号为右中括号前一个非终结符所在行号。	数组定义和使用中的 ']'
printf 中格式字符与表达式个数不匹配	l	报错行号为 'printf' 所在行号。	Stmt $\rightarrow$ 'printf' '(' FormatString '{', Exp } ')' ';'
在非循环块中使用 break 和 continue 语句	m	报错行号为 'break' 和 'continue' 所在行号。	Stmt $\rightarrow$ 'break' ';' 或 'continue' ';'

在词法分析过程中，如果遇到了单个'&'或'|'，之前的处理方式是报错（错误代码a）后直接 `break`，但这样无法继续输出如上面的完整结果。

```

if (currentChar == '&'){
    if (nextChar == '&'){
        op += inputFile.get();
        tokens.push_back({ AND, op, lineNumber });
    }

    else{
        errors.push_back({ lineNumber, 'a' });
        tokens.push_back({ AND, "&", lineNumber });
    }
}
```

```
        continue;
    }
}
```

当遇到单个'&'时，我会先输出错误信息，然后使用 `continue` 而不是 `break`，让代码继续执行。此外，因为错误已经被记录，所以我强制修复这个错误，将单个'&'自动补全为'&&'，确保代码可以正常执行。

这样处理后，即使出现单个'&'符号，程序仍能按预期继续运行，同时修正错误的符号。同理这样处理了'|'。

3. 语义分析

本次作业的主要目的是是通过管理作用域和符号表来检查程序的语义逻辑。

`全局变量` 作用域编号为1，局部作用域按嵌套递增，通过栈结构管理作用域关系；符号表记录变量、常量、函数等符号信息，支持唯一性检查和跨作用域引用。同时，语义分析需识别未定义符号、重复定义、类型不匹配等错误，并报告错误行号和类别码，为后续阶段提供支持。

以下是关于 `Symbol` 结构的构建以及设计理由的解释：

符号表设计的原因与实现

符号表的设计是为了记录程序中定义的变量、常量、函数等符号的详细信息。

`Symbol` 结构定义了每个符号的属性，以满足上述需求。

```
struct Symbol {
    int scopelineNumber = 0; // 符号所属的作用域编号（全局作用域为1，其他作用域递增）
    string name;             // 符号名称（如变量名、函数名）
    string type;             // 符号类型（如 "Int", "Char", "IntArray", "VoidFunc" 等）
    bool isConst = false;    // 是否为常量
    bool isArray = false;    // 是否为数组
    bool isFunction = false; // 是否为函数
    bool isInitialized = false; // 是否已被初始化
    int address = -1;        // 符号的内存地址（-1 表示尚未分配地址）
    int value = 0;           // 常量值或变量的默认值（适用于常量或已初始化的变量）
    int arraySize = 0;       // 数组大小（如果是数组类型）
};
```

然后通过，这段代码向符号表中添加一个新的符号（变量、函数等）。它会初始化符号的名称、类型、作用域编号、是否为常量、是否为数组、是否为函数等信息。如果符号不是函数，会分配内存地址（通过 `getNextAddress` 函数），然后将符号加入符号表 `symbolTable` 中，以便后续对符号的管理和引用。（`getNextAddress` 会用在代码优化部分）

```
// 向符号表中添加一个新的符号
void addSymbol(const string& name, const string& type, bool isFunction = false) {
    Symbol sym;
    sym.name = name; // 设置符号名称
    sym.type = type; // 设置符号类型
    sym.scopelineNumber = CurrentScope; // 设置作用域编号
    sym.isFunction = isFunction; // 是否为函数
    sym.isConst = false; // 默认非常量
    sym.isArray = false; // 默认非数组
    sym.value = 0; // 默认值为0

    if (!isFunction) {
        sym.address = getNextAddress(); // 分配内存地址
    }
    else {
        sym.address = -1; // 函数地址稍后分配
    }
}
```

```
symbolTable.push_back(sym); // 将符号加入符号表
}
```

在代码中，作用域管理主要通过以下函数和数据结构实现。

`enterScope` 函数在进入新作用域时调用，它将当前作用域的地址信息复制到 `nextAddressStack` 中，生成新的作用域编号 `scopeCounter`，并记录当前作用域的父作用域关系 `scopeParent`。

同时，将当前作用域压入 `scopeStack` 栈中，并更新 `CurrentScope` 为新的作用域编号。而 `exitScope` 函数在退出作用域时调用，它通过从 `scopeStack` 中弹出上一个作用域编号来恢复 `CurrentScope`，并从 `nextAddressStack` 中移除当前作用域的地址信息。

```
void enterScope() {
    nextAddressStack.push(nextAddressStack.top());
    scopeCounter++;
    scopeParent[scopeCounter] = CurrentScope;
    scopeStack.push(CurrentScope);
    CurrentScope = scopeCounter;
}

void exitScope() {
    if (!scopeStack.empty()) {
        CurrentScope = scopeStack.top();
        scopeStack.pop();
    }
    if (!nextAddressStack.empty()) {
        nextAddressStack.pop();
    }
}
```

最后通过该函数，可以统一处理错误，并将错误信息（行号和错误代码）存储到 `errors` 列表中。同时，通过 `errorLines` 集合确保每行只记录一次错误，避免重复处理。同一行多次错误不会重复添加，从而简化了错误管理逻辑。

```
void addError(int lineNumber, char errorcode) {
    if (errorLines.find(lineNumber) == errorLines.end()) {
        errors.push_back({ lineNumber, errorcode });
        errorLines.insert(lineNumber);
        cout << "DEBUG: Error '" << errorcode << "' added at line " << lineNumber << endl;
    }
    else {
        cout << "DEBUG: Error not added because line " << lineNumber << " already has an error." << endl;
    }
}
```

4. 代码生成

本次作业是，根据文法规则和语义约定，采用自顶向下的语法制导翻译技术，生成PCODE目标代码，并通过解释器执行生成的代码。

首先，需要添加 `Value` 类

在原有的解释器实现中，栈（`dstack`）仅能处理整数（`int`）类型的数据。然而，实际编程语言中变量不仅仅是整数，还可能包含字符串（`string`）等其他类型。为了增强解释器的功能，使其能够处理多种数据类型，我们引入了一个通用的 `Value` 类，该类可以存储整数或字符串，并通过类型标识来区分当前存储的数据类型。

```
// 定义一个通用的值类，可以存储整数或字符串
class Value {
private:
    int value;           // 存储整数值
```

```

    string Svalue;        // 存储字符串值
    bool isInt;           // 标识当前存储的类型：true为整数，false为字符串

public:
    // 整数值构造函数
    Value(int value) : value(value), isInt(true) {}

    // 字符串值构造函数
    Value(const string& Svalue) : Svalue(Svalue), isInt(false) {}

    // 返回整数值
    int getValue() const {
        if (isInt) {
            return value;
        }
        // 错误处理或返回默认值
        cerr << "错误：尝试从非整数Value获取整数值。" << endl;
        return 0;
    }

    // 返回字符串值
    string getSValue() const {
        if (!isInt) {
            return Svalue;
        }
        // 错误处理或返回默认值
        cerr << "错误：尝试从整数Value获取字符串值。" << endl;
        return "";
    }

    // 设置整数值
    void setValue(int value) {
        this->value = value;
        isInt = true;
    }

    // 设置字符串值
    void setSValue(const string& Svalue) {
        this->Svalue = Svalue;
        isInt = false;
    }

    // 判断是否为整数
    bool isInteger() const {
        return isInt;
    }

    // 判断是否为字符串
    bool isString() const {
        return !isInt;
    }

    // 重载输出运算符，以便直接输出Value对象
    friend ostream& operator<<(ostream& os, const Value& value) {
        if (value.isInt) {
            os << value.value;    // 输出整数值
        }
        else {
            os << "\"" << value.Svalue << "\"";    // 输出字符串值
        }
        return os;
    }
};

```

`Value` 类通过 `value`（整数）、`Svalue`（字符串）和 `isInt`（类型标识）三个私有成员变量来存储不同类型的数据，提供了相应的构造函数、访问和修改方法，以及重载的输出运算符，以便根据存储的类型正确处理和显示值。这一设计使得解释器能够灵活地处理多种数据类型，显著提升了其功能和可扩展性。

然后，`interpretPcode` 函数以支持 `Value` 类

为了使解释器能够处理多种数据类型（如整数和字符串），需要修改栈的类型以及相关的操作，使其能够存储和操作 `Value` 对象，而不仅仅是 `int`。这意味着所有栈操作、运算符处理和输入输出指令都需要相应地适配 `Value` 类。

```
// 修改后的栈类型，现在存储Value对象而非int
Value dstack[STACK_SIZE];
int sp = -1; // 栈指针初始化

// 其他全局变量，如 symbolTable, functionMap, etc.

// 定义一个向量来存储解释结果
vector<string> interpretResult;

// 假设已经定义了 callStack, nextAddressStack 等

void interpretPcode(const string& inputFile, const string& outputFileName) {

    nextAddressStack.push(0);

    ofstream outputFile(outputFileName);

    int pc = 0;      // 程序计数器
    int returnValue = 0;

    int variableSpace = nextaddress;
    sp = variableSpace - 1;
    // 全局标志，用于检查是否已输出
    bool hasPrinted = false;

    // 变量地址映射（与符号表保持一致）
    for (const Symbol& sym : symbolTable) {
        if (!sym.isFunction && sym.address != -1 && !sym.isConst) {
            string scopedName = to_string(sym.scopelineNumber) + "_" + sym.name;

            if (variables.find(scopedName) == variables.end()) {
                variables[scopedName] = sym.address;
            }
            else {
                cerr << "警告：" << sym.name << " 在作用域 " << sym.scopelineNumber << " 中重复定义。" << endl;
            }
        }
    }

    // 程序开始时设置main函数的位置
    if (functionMap.find("main") != functionMap.end()) {
        pc = functionMap["main"];
        cout << "\n开始执行主函数。PC: " << pc << endl;
    }

    // 解释器主循环
    while (pc < pcodeList.size()) {
        Pcode currentPcode = pcodeList[pc];
        string opcode = currentPcode.opcode;
        string operand = currentPcode.operand;

        if (opcode == "INT") {
            // 增加栈指针以分配空间
            int n = stoi(operand);
            sp += n;
            if (sp >= STACK_SIZE) {
```

```
        cerr << "错误：栈溢出在 PC " << pc << endl;
        break;
    }
    pc++;
}
}
```

为了支持多种数据类型（如整数和字符串），我们将原先的整数栈 `int dstack[STACK_SIZE];` 修改为 `Value dstack[STACK_SIZE];`，使栈能够存储 `Value` 对象。这一改动允许解释器在执行诸如 `ADD`、`SUB` 等算术运算时，首先从栈中弹出两个 `Value` 对象，并通过 `isInt` 标识检查它们是否为整数类型，仅在两者均为整数时才执行相应运算并将结果作为新的 `Value` 对象压入栈中。如果尝试对非整数类型执行运算，解释器将输出错误信息并停止执行。此外，`LIT` 指令被修改为将操作数转换为整数后创建 `Value` 对象并推入栈中，而 `READ_INT` 和 `READ_CHAR` 指令则分别读取整数和字符输入，并封装为 `Value` 对象后推入栈中。在输出指令方面，`WRT` 根据表达式数量从栈中获取相应的 `Value` 对象并根据其类型进行输出，`WRTS` 则直接输出字符串常量，同时处理引号和换行符。在所有操作之前，均进行类型检查以确保操作的合法性，若类型不匹配或操作不支持，解释器将输出错误信息并根据需要终止执行。

此外，为了全面支持 `Value` 类，还需要对栈操作函数（如 `STO`、`LOD` 等）进行相应修改，使其处理 `Value` 对象而非简单的整数。运算符扩展也成为必要，除了基本的算术运算符外，可能需要添加字符串连接等操作，并引入类型转换指令以在不同类型之间进行转换，例如将整数转换为字符串或反之。符号表与变量管理部分也需调整，使其能够存储 `Value` 对象，以支持多种数据类型。通过这些改动，解释器不仅提升了处理多种数据类型的能力，增强了其灵活性和功能性，还显著提高了代码的可扩展性，为未来添加更多数据类型奠定了坚实基础。

虽然，我还没做完。这可能逻辑上会有不少的漏洞。以及我觉得sp（STACK POINTER）还是需要再调整。但到最后都还没找到了是什么原因，，但这次的编译实验对我还是会有很大的帮助！